what are the key differences between the two solutions

High-Level Difference

Although both programs use Linux TUN interfaces and manipulate packets manually, they operate at **different networking layers** and demonstrate **different architectural goals**.

Core Conceptual Difference

Program	Main Purpose
First program	Layer-2 Ethernet tunneling
Second program	Layer-3 protocol stack simulation

Networking Layer Difference

First Program → Layer 2 (Ethernet)

The first program:

- creates raw Ethernet frames
- manipulates MAC addresses
- uses raw packet sockets (AF_PACKET)
- adds/removes Ethernet headers manually

It behaves like:

- a primitive Ethernet tunnel
- a custom Layer-2 transport

Packet Format

```
+------------------+
| Ethernet Header  |
+------------------+
| IPv4 Packet      |
+------------------+
```

Second Program → Layer 3 (IP)

The second program:

- only handles IP packets
- never touches Ethernet headers
- relies on Linux routing

- focuses on protocol architecture

It behaves more like:

- a mini userspace protocol stack
- a packet-processing engine

Packet Format

```
+-------------------+
| IPv4 Header       |
+-------------------+
| Payload           |
+-------------------+
```

TUN Usage Difference

First Program

Uses TUN as:

IP packet source/sink

but then converts packets into:

raw Ethernet frames

So TUN is merely the ingress/egress point.

Second Program

Uses TUN directly as:

the entire transport mechanism

No Ethernet exists in userspace at all.

Linux handles routing automatically.

Kernel Interaction Difference

First Program

Interacts with:

- /dev/net/tun
- raw packet sockets
- Ethernet layer

- NIC driver layer

Uses:

socket(AF_PACKET, SOCK_RAW, ...)

This bypasses much of the normal networking stack.

Second Program

Interacts only with:

- /dev/net/tun
- Linux IP routing stack

No raw sockets.

No direct NIC interaction.

Encapsulation Difference

First Program

Performs explicit encapsulation:

IP packet
  ↓
wrap in Ethernet
  ↓
send raw frame

and decapsulation on RX.

This is true tunneling behavior.

Second Program

Does not encapsulate anything.

It simply:

constructs IP packet
  ↓
injects into TUN

Linux handles forwarding.

Addressing Difference

First Program

Uses:

- MAC addresses
- EtherType
- Ethernet frame routing

Needs:

destination MAC

to transmit.

Second Program

Uses only:

- IP addresses
- Linux routing table

No MAC awareness exists in the application.

Socket Difference

First Program	Second Program
Raw packet socket	No sockets beyond TUN FD
AF_PACKET	none
Ethernet frames	IP packets only

Routing Difference

First Program

Implements its own Layer-2 delivery mechanism.

The application decides:

- frame destination
- encapsulation

Linux routing is mostly bypassed.

Second Program

Relies entirely on Linux routing:

ip route replace ...

Linux decides:

- forwarding
- ARP
- next-hop resolution

Protocol Abstraction Difference

First Program

Is transport-oriented.

Focus:

- moving packets between interfaces

Little protocol abstraction exists.

Second Program

Is architecture-oriented.

Focus:

- simulating kernel networking structures:
    - mbuf
    - ifnet
    - protocol input/output

This resembles OS internals.

Educational Focus Difference

First Program Teaches	Second Program Teaches
Ethernet framing	Protocol stack structure
Raw sockets	Packet abstraction

First Program Teaches	Second Program Teaches
EtherTypes	IPv4 header creation
Layer-2 tunneling	Kernel-style networking
Encapsulation	Packet processing

Security Difference

First Program

Potentially dangerous because:

- raw Ethernet injection
- custom EtherTypes
- direct NIC access

Requires:

- root privileges
- raw socket capability

Could interfere with LAN traffic.

Second Program

Much safer:

- isolated to TUN interfaces
- uses normal kernel routing

Closer to ordinary virtual networking.

Complexity Difference

First Program

More complex networking-wise because it handles:

- Ethernet headers
- MAC addresses
- EtherTypes

- raw frame transmission

Closer to driver-level networking.

Second Program

More complex architecturally because it introduces:

- protocol abstractions
- packet structures
- stack layering concepts

Closer to OS networking internals.

Real-World Analogy

First Program

Comparable to:

- GRE-like tunneling
- L2TP concepts
- Ethernet-over-IP designs
- simple VPN transport

Second Program

Comparable to:

- userspace protocol stacks
- router software internals
- network simulators
- protocol research frameworks

Data Path Comparison

First Program

TUN
↓
Userspace

↓
Ethernet encapsulation
↓
Raw socket
↓
NIC

Second Program

TUN
↓
Userspace protocol logic
↓
TUN
↓
Kernel routing

Key Architectural Insight

The first program treats Linux as:

a packet source and sink

The second program treats Linux as:

a routing and forwarding engine

Most Important Difference

First Program

Implements:

custom transport over Ethernet

Second Program

Implements:

custom protocol processing over IP

Final Comparison

Aspect	**First Program**	**Second Program**
Layer	Layer 2	Layer 3

Aspect	First Program	Second Program
Uses Ethernet headers	Yes	No
Uses MAC addresses	Yes	No
Uses raw sockets	Yes	No
Uses routing table	Minimally	Extensively
Encapsulation	Ethernet encapsulation	None
Main concept	Packet tunneling	Protocol stack simulation
Focus	Transport mechanics	Protocol architecture
Kernel interaction	NIC-level	Routing-level
Educational theme	Ethernet networking	OS networking internals
Complexity style	Networking complexity	Architectural complexity